```
!pip install wfdb
```

Mostrar el resultado oculto

Concepto clave

PhysioNet organiza los datos como:

- record → señal (ej: ECG)
- annotation → etiquetas (ej: latidos, arritmias)
- metadata → frecuencia de muestreo, unidades, etc.

Descargar y cargar un registro

Ejemplo con el dataset clásico:

- MIT-BIH Arrhythmia Database

```
import wfdb

# Descargar un registro (ej: 100)
record = wfdb.rdrecord('100', pn_dir='mitdb')

# Read a WFDB anno tation file record_name.extension and return an Annotation
annotation = wfdb.rdann('100', 'atr', pn_dir='mitdb')

# Tipo: numpy array
# Forma: (N, canales)
signal = record.p_signal
fs = record.fs
```

```
import wfdb
```

```
wfdb.get_dbs()
```

```
---------------------------------------------------------------------------
NetFileNotFoundError                      Traceback (most recent call last)
/tmp/ipykernel_49576/10376211.py in <cell line: 0>()
----> 1 wfdb.get_dbs()

                              4 frames
/usr/local/lib/python3.12/dist-packages/wfdb/io/_url.py in _parse_headers(self, method, response)
    211             else:
    212                 cls = NetFileError
--> 213             raise cls(
    214                 "%s Error: %s for url: %s"
    215                 % (response.status_code, response.reason, response.url),

NetFileNotFoundError: 404 Error: Not Found for url: https://physionet.org/rest/database-list/
```

```
help(wfdb.io)
```

The annotation.symbol attribute provides a list of classifications for each detected event (usually a heartbeat) in the ECG signal. For the MIT-BIH Arrhythmia Database, these symbols have specific meanings:

- N: Normal beat
- +: R-on-T premature ventricular contraction (not a beat, but marks a rhythm change or non-beat event, in this context specifically used as lead switch)
- A: Atrial premature beat

There are many other symbols representing various types of beats and events (e.g., V for premature ventricular contraction, ! for ventricular flutter wave). A comprehensive list is usually available in the documentation for the specific PhysioNet database you are using. These annotations are crucial for identifying and studying different cardiac arrhythmias.

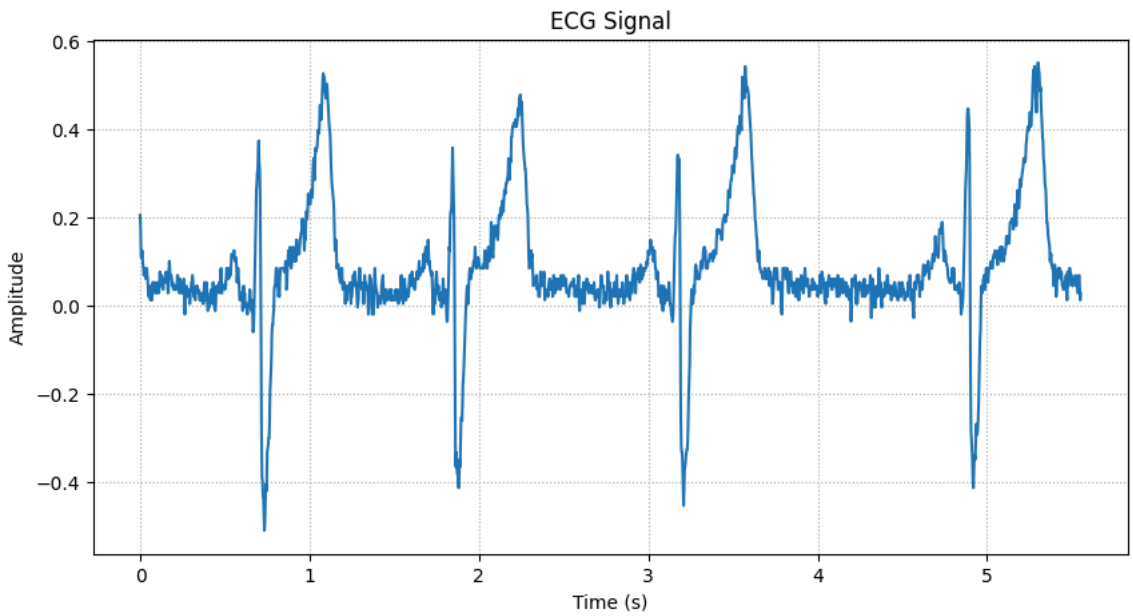
```
# https://archive.physionet.org/physiobank/database/records.shtml
# recordx = wfdb.rdrecord('118e00', pn_dir='nstdb')
# signalx = recordx.p_signal
# plt.plot(signalx[:2000,0])
```

```
annotation.sample, annotation.symbol, annotation.aux_note
```

```
signal, signal.shape
```

```
(array([[ 0.20502, -0.02192, -0.20534, ...,  0.32206,  0.13324, -0.07612],
        [ 0.17286, -0.05116, -0.21658, ...,  0.30832,  0.11794, -0.09096],
        [ 0.10854, -0.07552, -0.1997 , ...,  0.2963 ,  0.1092 , -0.1021 ],
        ...,
        [ 0.62314,  0.00242, -0.46408, ...,  0.53164,  0.32328,  0.14292],
        [ 0.59098,  0.00242, -0.44158, ...,  0.52476,  0.32328,  0.14664],
        [ 0.56686,  0.00242, -0.4247 , ...,  0.51962,  0.32546,  0.15036]]),
 (5000, 12))
```

```
import matplotlib.pyplot as plt
import numpy as np

N = signal.shape[0]
n = np.arange(N)
t = n/fs

plt.figure(figsize=(10,5))
plt.plot(t[:2000], signal[:2000,0])
plt.title("ECG Signal")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid(ls=":")
plt.show()
```




```
x = signal[:,0]

plt.magnitude_spectrum(x, Fs=fs, scale='dB')
```

```
x_dc_removed = x - np.mean(x)

plt.magnitude_spectrum(x_dc_removed, Fs=fs, scale='dB')
```

```
X = np.fft.fft(x_dc_removed)
freqs = np.fft.fftfreq(len(x_dc_removed), 1/fs)

plt.plot(freqs[:len(x_dc_removed)//2], np.abs(X[:len(x_dc_removed)//2]))
plt.xlabel("Hz")
plt.ylabel("Magnitude")
plt.show()
```

```
wfdb.plot_wfdb(record=record, annotation=annotation)
```

filtro pasa banda con

```
from scipy.signal import butter, filtfilt

def bandpass(signal, fs, low=0.5, high=40):
```

```
    b, a = butter(4, [low/(fs/2), high/(fs/2)], btype='band')
    return filtfilt(b, a, signal)

filtered = bandpass(signal[:,0], fs)
```

```
X = np.fft.fft(filtered)
freqs = np.fft.fftfreq(len(filtered), 1/fs)

plt.plot(freqs[:len(filtered)//2], np.abs(X[:len(filtered)//2]))
plt.xlabel("Hz")
plt.ylabel("Magnitude")
plt.show()
```

Segmentación (ventanas)

```
window_size = int(2 * fs)  # 2 segundos
segments = []

for i in range(0, len(filtered)-window_size, window_size):
    segments.append(filtered[i:i+window_size])
```

Feature extraction

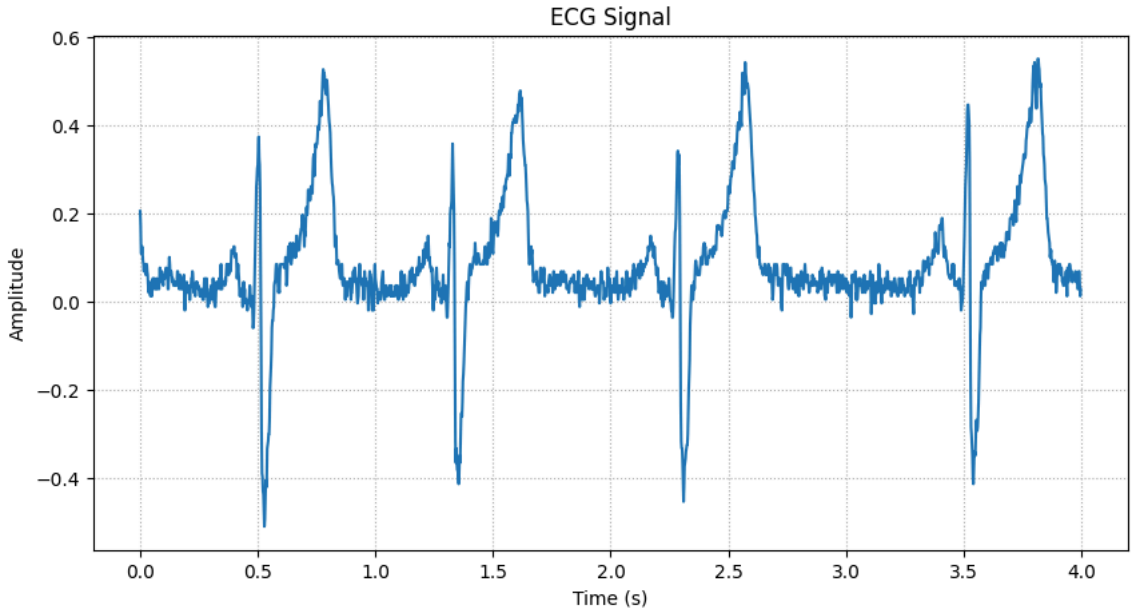
```
def extract_features(segment):
    return [
        np.mean(segment),
        np.std(segment),
        np.max(segment)
    ]
```

```
extract_features(segments[0])
```

```
plt.plot(segments[0])
```

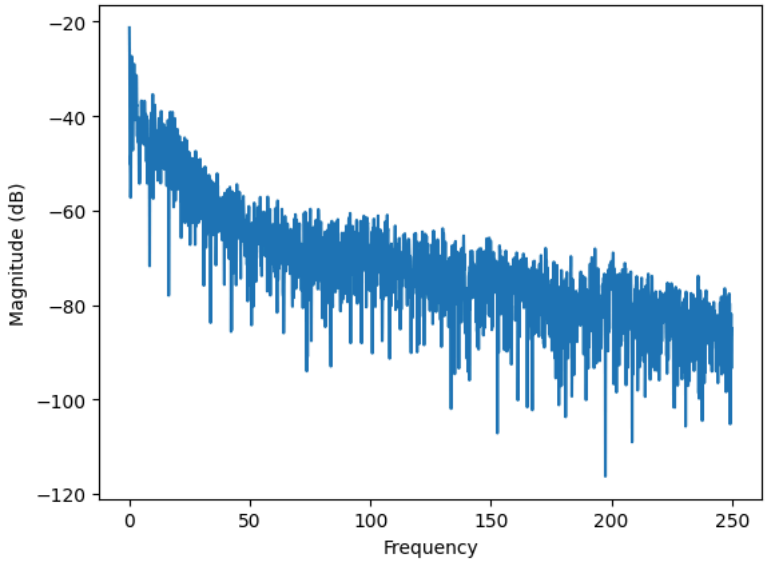
```
## Ejercicio con datos de Physionet
import wfdb

#Ver nombres de registros
records = wfdb.get_record_list('norwegian-athlete-ecg')
print(records)

#Cargar uno
record = wfdb.rdrecord(records[0], pn_dir='norwegian-athlete-ecg/1.0.0')

signal = record.p_signal
fs = record.fs
N = signal.shape[0]   # número de muestras
n = np.arange(N)
t = n / fs            # tiempo en segundos

# Graficar
plt.figure(figsize=(10,5))
plt.plot(t[:2000], signal[:2000,0])  # primeros 2000 puntos, canal 0
plt.title("ECG Signal")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid(ls=":")
plt.show()
```

```
['ath_001', 'ath_002', 'ath_003', 'ath_004', 'ath_005', 'ath_006', 'ath_007', 'ath_008', 'ath_009', 'ath_010', 'ath_011', 'a
```



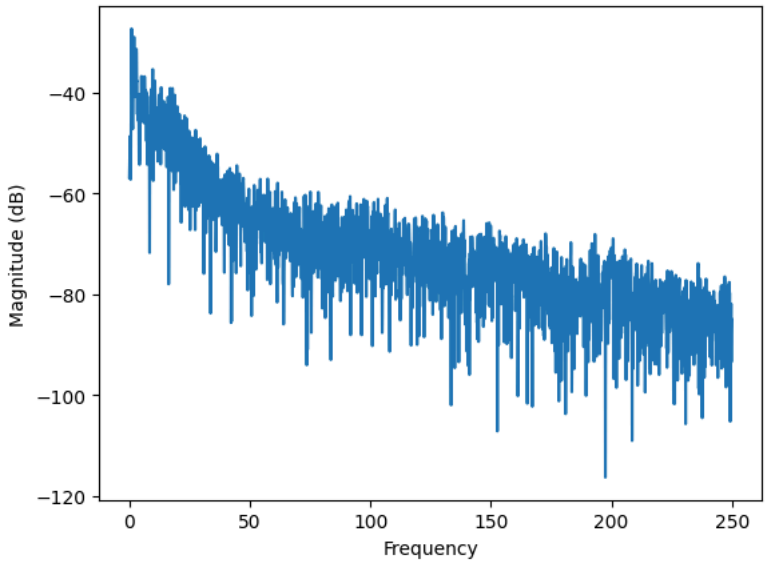

```
x = signal[:,0]

plt.magnitude_spectrum(x, Fs=fs, scale='dB')
```

```
(array([8.65731127e-02, 4.29593045e-02, 3.16038860e-03, ...,
        2.12639797e-05, 5.45276197e-05, 5.62668479e-05]),
 array([0.000e+00, 1.000e-01, 2.000e-01, ..., 2.498e+02, 2.499e+02,
        2.500e+02]),
 <matplotlib.lines.Line2D at 0x7aa4ab843cb0>)
```

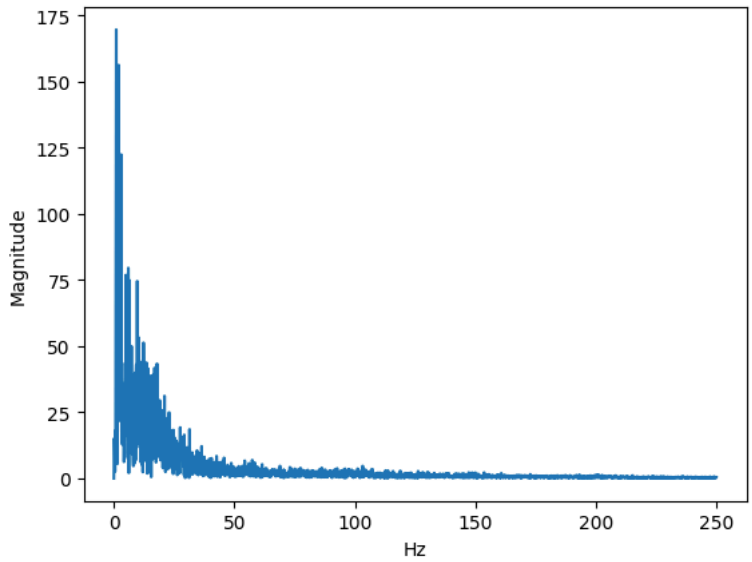



```
x_dc_removed = x - np.mean(x)

plt.magnitude_spectrum(x_dc_removed, Fs=fs, scale='dB')
```

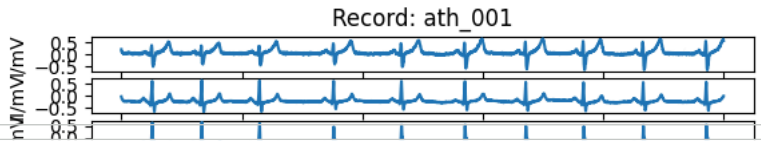
```
(array([1.44007668e-03, 1.99080170e-03, 3.16487315e-03, ...,
        2.12639797e-05, 5.45276197e-05, 5.62668479e-05]),
 array([0.000e+00, 1.000e-01, 2.000e-01, ..., 2.498e+02, 2.499e+02,
        2.500e+02]),
 <matplotlib.lines.Line2D at 0x7aa4ab7dd6d0>)
```



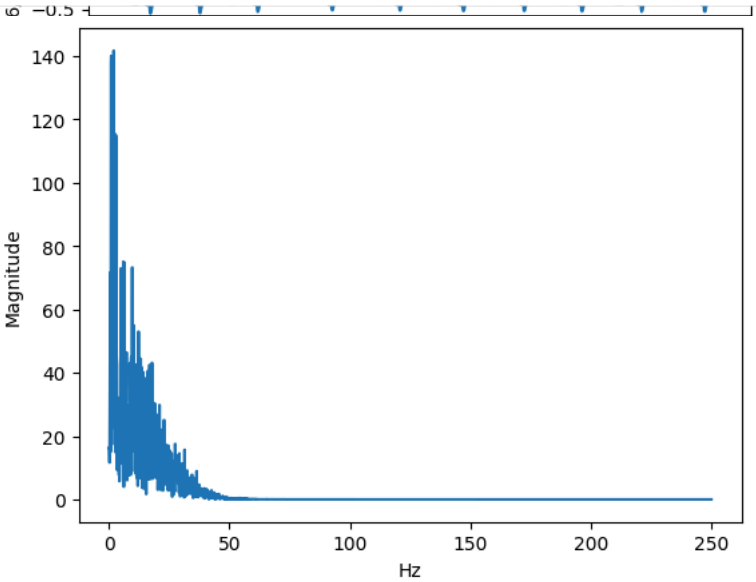

```
X = np.fft.fft(x_dc_removed)
freqs = np.fft.fftfreq(len(x_dc_removed), 1/fs)

plt.plot(freqs[:len(x_dc_removed)//2], np.abs(X[:len(x_dc_removed)//2]))
plt.xlabel("Hz")
plt.ylabel("Magnitude")
plt.show()
```

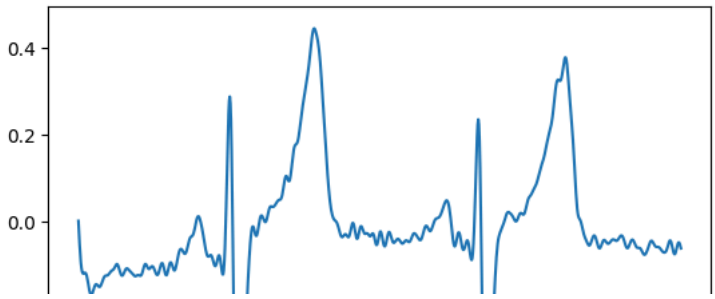



```
wfdb.plot_wfdb(record=record)
```




```
from scipy.signal import butter, filtfilt

def bandpass(signal, fs, low=0.5, high=40):
    b, a = butter(4, [low/(fs/2), high/(fs/2)], btype='band')
    return filtfilt(b, a, signal)

filtered = bandpass(signal[:,0], fs)
X = np.fft.fft(filtered)
freqs = np.fft.fftfreq(len(filtered), 1/fs)

plt.plot(freqs[:len(filtered)//2], np.abs(X[:len(filtered)//2]))
plt.xlabel("Hz")
plt.ylabel("Magnitude")
plt.show()
```




```
window_size = int(2 * fs)  # 2 segundos
segments = []

for i in range(0, len(filtered)-window_size, window_size):
    segments.append(filtered[i:i+window_size])
def extract_features(segment):
    return [
        np.mean(segment),
        np.std(segment),
        np.max(segment)
    ]
extract_features(segments[0])
plt.plot(segments[0])
```

```
[<matplotlib.lines.Line2D at 0x7aa4a91699d0>]
```